AI ENGINEERING BOOTCAMP

Tools, Topics and Techniques Learned Across the Resources and Assignments

This PDF consolidates the bootcamp resources into one study map. It covers the main technical topics, tools, engineering practices, and project techniques from the weekly lessons, slide decks, and assignments. It is written as a review document for presentations, interviews, and final project defense.

The bootcamp arc moves from ML foundations to LLM applications, then to RAG, agents, MLOps, production services, eval-driven development, advanced retrieval, and finally multi-tenant AI SaaS architecture.

Source Coverage

Resource group	Examples from the project resources	What they contributed
Week 1	Intro to ML, supervised learning, starter toolkit	Core ML concepts, datasets, training, testing, metrics, model workflow
Week 2	AWS, Docker, hosted LLMs, prompt engineering	Cloud basics, containers, API-based LLM usage, prompting patterns
Week 3	RAG lessons, vector stores, RAGAS, Decision Intelligence Assistant	Embeddings, semantic search, RAG pipeline, retrieval evaluation, ML vs LLM comparison
Week 4	Agentic AI lessons, Smart Travel Planner	Tool calling, planning, agent loops, safety, scoped tools, RAG with agents
Week 5	MLOps project, queues, MCP, A2A, Spec Kit	Drift monitoring, model registry, Redis queues, human approval, protocols, specifications
Week 6	Eval-driven development, document classifier service, NeMo Guardrails	Evals, golden tests, auth service architecture, guardrails, caching, workers, artifact discipline
Week 7	Maintainer's Copilot, advanced RAG, GraphRAG, reranking	Fine-tuning, advanced retrieval, memory, widgets, CI eval gates
Week 8	Concierge, state/correctness, system structuring	Multi-tenancy, tenant isolation, transactions, outbox, clean boundaries, SaaS architecture

1. The Complete Tool Map

Category	Tools / Platforms	What they were used for
Languages & notebooks	Python, Google Colab, Jupyter notebooks	Experimentation, training, evaluation, dataset preparation, artifact export
Data science	NumPy, pandas, scikit-learn	Data loading, preprocessing, classical ML baselines, metrics, train/test splits
Deep learning / NLP	PyTorch, Transformers, DistilBERT, ConvNeXt, ONNX, ONNX Runtime	Fine-tuning, document image classification, text classification, model export, lightweight serving
LLM providers	Hosted LLM APIs such as Groq/OpenAI-style APIs	Prompting, summarization, extraction, reasoning, agents, LLM baselines
Prompting	Zero-shot, few-shot, CoT, role prompting, constraints, output schemas	Controlling model behavior and making outputs predictable
RAG	Embeddings, ChromaDB, pgvector, vector stores, rerankers, RAGAS	Answering from private/project documents with retrieval and evaluation
Agents	LangChain, LangGraph, ReAct, tool calling, bounded loops	Systems that choose tools, act, persist state, and recover
Backend	FastAPI, Pydantic, SQLAlchemy, Alembic	APIs, validation, database access, migrations, typed contracts
Frontend	Streamlit, React widget, embeddable web component	Dashboards, chatbot UI, tenant admin UI, public website widget
Storage	Postgres, pgvector, MinIO/S3-style blobs	Structured data, vector search, document/model/artifact storage
Infrastructure	Docker, Docker Compose, AWS basics	Local reproducible services and cloud deployment concepts
Security	JWT, RBAC, Casbin, Row-Level Security, Vault	Authentication, authorization, tenant boundaries, secrets management
Queues & workers	Redis, RQ/Celery-style workers, Kafka concepts	Slow tasks, asynchronous jobs, event streaming, resilient background processing
Quality & CI	pytest, ruff, eval scripts, golden sets, GitHub Actions	Automated tests, code quality, regression checks, eval gates
Observability	Logs, traces, metrics, dashboards, redaction	Debugging, auditability, privacy-safe production visibility
Guardrails	NeMo Guardrails, sidecar guardrail service	Input/output safety, policy checks, grounded responses, refusal paths
Collaboration	Git, GitHub branches, pull requests, Trello	Team workflow, review, task tracking, delivery evidence

2. Week-by-Week Knowledge Summary

Week 1 - Machine Learning Foundations

- What ML is: learning patterns from data instead of hard-coding rules.
- Supervised learning: input features, labels, training data, test data, predictions.
- Classification vs regression and when to use each.
- Dataset preparation: cleaning, splitting, feature selection, encoding, scaling.
- Metrics: accuracy, precision, recall, F1, confusion matrix, train/test performance.
- Starter toolkit: Python, notebooks, pandas, NumPy, scikit-learn, basic visualization.

Main outcome: Given a labeled dataset, train a baseline model, evaluate it, explain errors, and improve the pipeline without guessing.

Week 2 - Cloud, Containers, Hosted LLMs and Prompt Engineering

- Docker fundamentals: image, container, Dockerfile, volumes, ports, compose services.
- Why containers solve dependency problems and make local development repeatable.
- AWS/cloud basics: compute, storage, networking, deployment thinking, managed services.
- Hosted LLM architecture: app sends prompt/API request; provider runs model on GPUs.
- Prompt engineering: zero-shot, few-shot, role, constraints, examples, output formatting.
- Prompt reliability: be specific, provide context, define labels, include edge cases, request structured output.

Main outcome: Wrap an app in Docker, call a hosted LLM by API, and design prompts that return usable output.

Week 3 - RAG, Embeddings, Vector Search and Evaluation

- RAG motivation: avoid hallucination by retrieving real context before generation.
- Document loading and chunking: split large documents into searchable passages.
- Embeddings: turn text into vectors where semantic similarity becomes distance.
- Vector stores: store embeddings and retrieve relevant chunks using similarity search.
- RAG prompt: combine question + retrieved context + instructions + source attribution.
- RAG evaluation: separate retrieval failure from generation failure using RAGAS-style metrics.
- Assignment focus: Decision Intelligence Assistant with RAG, source panel, ML baseline, LLM baseline, cost/latency comparison.

Main outcome: Build a grounded assistant that answers from documents and measure whether failures come from retrieval or generation.

Week 4 - Agentic AI and Tool-Using Systems

- Agentic AI: LLMs that do not only answer, but choose actions and tools.
- Tool calling: expose functions/APIs with schemas so the model can call them safely.
- ReAct-style loop: think, choose action, observe result, continue until answer.
- Planner/executor pattern: split a task into steps, then execute with tools.
- Human-in-the-loop: require approval when actions are risky or irreversible.
- Safety basics: scoped tools, allowlists, bounded loops, traceability, fallback behavior.
- Assignment focus: Smart Travel Planner using agents, tools, RAG, and structured planning.

Main outcome: Design an agent that can search, plan, call tools, and stop safely instead of looping forever.

Week 5 - Production Agents, MLOps, Queues, MCP/A2A and Specs

- MLOps lifecycle: train, validate, register, serve, monitor, retrain, rollback.
- Drift: production data changes over time, so model performance can silently decay.
- Redis queues: move slow tasks out of request/response flow for reliability and latency.
- LangGraph supervisor: stateful orchestration where an agent can resume after failure.
- Human approval gates: production model changes should not happen automatically.
- MCP: standard way for agents to connect to tools without custom glue everywhere.
- A2A: agent-to-agent communication pattern for multi-agent systems.
- Spec-driven development: write requirements/contracts so AI-assisted coding has a target.

Main outcome: Build a drift triage co-pilot that detects a signal, queues work, asks for approval, and changes production only through a controlled path.

Week 6 - Authenticated Services, Document Classification, Guardrails and Evals

- Eval-driven development: define metrics and datasets before changing prompts/models.
- Golden tests: fixed test cases that must remain stable across changes.
- Document classification: classify image/document layouts such as invoices, resumes, forms, letters.
- Service architecture: FastAPI API, service layer, repository layer, domain schemas, DB models.
- Authentication and authorization: JWT users, RBAC, permission-gated APIs.
- Infrastructure services: Postgres, Redis, MinIO, Vault, SFTP ingest, workers.
- Caching and invalidation: speed up reads but clear cache when state changes.
- Guardrails: detect unsafe/off-topic/private-data behavior before/after LLM calls.

Main outcome: Turn a model into an authenticated internal service with real ingestion, permissions, workers, cache, artifacts, and CI checks.

Week 7 - Maintainer's Copilot and Advanced RAG

- Fine-tuned issue classifier: bug, feature, docs, question labels on open-source issues.
- Entity extraction and summarization: pull structured facts and summarize long threads.
- Advanced RAG: parent-document retrieval, multi-vector retrieval, HyDE, multi-query, agentic RAG.
- Reranking: retrieve many candidates, then reorder so the best chunks reach the LLM.
- GraphRAG: use relationships/entities to improve retrieval across connected knowledge.
- RAG failure modes: low-confidence retrieval, missing sources, irrelevant chunks, temporal questions.
- Authenticated chatbot with memory and embeddable widget.
- CI eval gates: fail builds when model/RAG quality drops below thresholds.

Main outcome: Create a copilot that classifies issues, retrieves project knowledge, remembers context, answers with sources, and is evaluated automatically.

Week 8 - Multi-Tenant AI SaaS, State, Correctness and Clean Boundaries

- Multi-tenancy: many businesses share one platform but must never see each other's data.
- Tenant isolation: tenant_id everywhere, RLS/authorization checks, scoped vector search, scoped blobs.
- Concierge SaaS: tenant admin CMS + public widget + agent that can answer and act.
- Classifier router: easy messages go to direct workflow; hard messages go to agent loop.
- Tools: rag, capture_lead, escalate, and other tenant-scoped actions.
- Model serving: lightweight model server using ONNX Runtime/sklearn; avoid heavy training dependencies in containers.
- State correctness: transactions, avoiding dual writes, outbox pattern, idempotency.
- System structure: monolith vs microservices, clean boundaries, sidecars, queues, event streaming concepts.

Main outcome: Defend a production-style SaaS architecture where tenants are isolated, state remains correct, and agents act only within safe boundaries.

3. Core Techniques You Should Be Able to Explain

Technique	Simple explanation	Why it matters in projects
Train/test split	Keep unseen data for evaluation.	Prevents claiming success only because the model memorized training data.
Baseline model	Start with a simple measurable model.	Gives a reference point before using expensive or complex models.
Class imbalance handling	Use class weights, sampling, or careful metrics.	Accuracy can lie when one class dominates.
Macro vs weighted F1	Macro treats classes equally; weighted follows class frequency.	Shows whether minority classes are being ignored.
Prompt constraints	Tell the LLM exact task, format, labels, and limits.	Reduces vague, inconsistent, or unusable answers.
Few-shot prompting	Show examples of desired input/output behavior.	Improves consistency for classification/extraction tasks.
Structured output	Ask for JSON, labels, tables, or schemas.	Makes LLM output easier to validate and use in code.
Chunking	Split documents into retrievable pieces.	Bad chunks cause missing context or noisy answers.
Embeddings	Represent text as numeric meaning vectors.	Allows semantic search, not just keyword matching.
Similarity search	Find chunks closest to a query vector.	Core retrieval step in RAG.
Reranking	Reorder retrieved candidates with a stronger judge.	Gets the most relevant chunks into the final prompt.
Source attribution	Show which retrieved documents supported the answer.	Builds trust and makes answers auditable.
RAGAS-style evaluation	Measure retrieval quality and answer faithfulness separately.	Tells whether to fix retriever or generator.
Tool schema design	Define tool name, parameters, types, and descriptions.	Helps agents call tools correctly.
Bounded loop	Limit how many tool steps an agent may take.	Prevents infinite loops and runaway cost.
Human-in-the-loop	Pause for approval before risky actions.	Prevents unsafe automatic production changes.
Queueing	Place slow tasks in a background work queue.	Keeps APIs responsive and survives spikes.
Model registry	Store model versions, metadata, and promotion state.	Supports rollback, approval, and reproducible deployment.
Golden set	Fixed examples with expected outputs.	Catches regressions after code/model changes.
Eval gate	CI check that blocks bad quality changes.	Turns AI quality into a measurable engineering requirement.
Guardrails	Policy layer around LLM input/output/tool actions.	Reduces jailbreaks, leaks, hallucination, and off-topic behavior.
RLS / tenant filtering	Database-level or app-level tenant isolation.	Critical for multi-tenant SaaS safety.
Transactions	Commit related DB writes together or not at all.	Prevents half-finished state.
Outbox pattern	Write event intent inside the same DB transaction, then publish later.	Avoids dual-write inconsistency.

4. Project and Assignment Map

Assignment	Main build	Important tools	Skills practiced
Week 1	ML practice and foundations	Python, pandas, scikit-learn, notebooks	Data prep, training, evaluation, baseline thinking
Week 2	Container/cloud/LLM exercises	Docker, AWS concepts, hosted LLM APIs	Reproducible environments, cloud awareness, prompt engineering
Week 3	Decision Intelligence Assistant	RAG, vector store, ML model, LLM baseline, Docker	Grounded QA, source panels, cost/latency comparison
Week 4	Smart Travel Planner	Agents, tools, RAG, planner/executor	Tool calling, workflow design, scoped actions
Week 5	Drift Triage Co-Pilot	Bank Marketing dataset, LangGraph, Redis queue, dashboard	MLOps, drift response, registry, approval, rollback/retrain
Week 6	Document Classifier Service	RVL-CDIP, FastAPI, JWT, Casbin, Postgres, Redis, MinIO, Vault	Authenticated service, workers, cache, artifacts, golden tests
Week 7	Maintainer's Copilot	DistilBERT/ML/LLM comparison, pgvector, Streamlit, widget, RAGAS	Fine-tuning, advanced RAG, memory, eval CI, chatbot deployment
Week 8	Concierge Multi-Tenant SaaS	FastAPI, tenant CMS, public widget, ONNX/sklearn model server, guardrails, traces	Tenant isolation, agent actions, correctness, SaaS architecture

5. Architecture Patterns Learned

Pattern	Where it appeared	Meaning
Layered backend	Week 6/7/8 services	Routers accept requests; services own business logic; repositories own SQL; schemas define contracts.
Model server	Week 7/8	Serve inference behind an API so the app does not carry training dependencies.
Sidecar service	Guardrails	A separate service handles a cross-cutting concern such as safety checks.
Worker pipeline	SFTP ingest, retraining, replay jobs	Background workers process slow tasks outside the web request.
Event/queue architecture	Redis/Kafka concepts	Move work through messages for reliability and scale.
Multi-tenant boundary	Concierge	Every query, tool, vector search, file, and log is scoped to one tenant.
Embeddable widget	Maintainer's Copilot and Concierge	A public UI can be embedded into a host app while backend state remains controlled.
Eval-driven delivery	Weeks 6-7	Quality thresholds and golden sets become part of CI, not manual judgment.

6. What to Say in a Defense or Interview

A strong defense answer should connect the topic to a concrete project file, system boundary, metric, or failure mode. The goal is not only to define the tool, but to explain why it was chosen and what could go wrong without it.

Topic	Strong answer angle
Why RAG?	Because the LLM alone does not know private documents. RAG retrieves trusted context first, then the LLM answers from that context with sources.
Why advanced RAG?	Naive chunk retrieval fails when the right evidence is split, buried, or semantically indirect. Parent-doc, multi-query, HyDE, GraphRAG, and reranking fix different retrieval failures.
Why fine-tune/classify?	A small specialized model is cheaper, faster, and more predictable for repeated classification than asking an LLM every time.
Why queues?	Slow or unreliable work should not block the user's HTTP request. Queues improve latency, retry behavior, and resilience.
Why guardrails?	Production LLM systems need explicit rules for off-topic input, unsafe output, data leakage, and tool misuse.
Why tenant isolation?	A multi-tenant SaaS is only safe if every API, DB row, vector, file, cache key, and log is scoped to the correct tenant.
Why evals?	AI behavior is probabilistic. Evals turn quality into measurable pass/fail checks so changes do not rely on vibes.
Why Docker?	It packages app dependencies and infrastructure so the system runs consistently across machines.
Why ONNX/runtime separation?	Training libraries are heavy. Exporting a model lets production run lean inference without carrying the full training stack.
Why transactions/outbox?	They prevent partially committed state and reduce the risk that the DB and external systems disagree after a crash.

7. Quick Glossary

Term	Meaning
RAG	Retrieval-Augmented Generation: retrieve relevant context, then generate an answer from it.
Embedding	A vector representation of text meaning.
Vector store	Database/index that stores vectors and supports similarity search.
Reranker	A second-stage model/judge that reorders retrieved chunks.
GraphRAG	Retrieval that uses graph relationships between entities/documents.
Agent	An LLM-driven workflow that can choose tools and act.
Tool calling	A model selecting a function/API call with structured arguments.
Bounded loop	An agent loop with a maximum number of steps.
Drift	Change in production data or behavior that can reduce model quality.
Model registry	A controlled store of model versions and deployment states.
Golden set	Fixed examples used to detect regressions.
Guardrail	Policy/control layer that constrains LLM behavior.
RLS	Row-Level Security: database rules that restrict which rows a user/tenant can see.
Outbox	A pattern that stores events in the DB transaction before publishing them externally.
MCP	Model Context Protocol: a standard way for agents to connect to tools/context.
A2A	Agent-to-Agent communication pattern.

8. Final Condensed Checklist

- I can explain the difference between classical ML, deep learning, and hosted LLMs.
- I can train, evaluate, and compare models using meaningful metrics.
- I can explain why prompt engineering needs specificity, examples, constraints, and structured output.
- I can build the RAG loop: load, chunk, embed, store, retrieve, prompt, answer, cite.
- I can diagnose RAG failures as retrieval failures or generation failures.
- I can design an agent with tools, bounded loops, state, safety, and human approval.
- I can explain queues, workers, model registries, drift, rollback, and retraining.
- I can design an authenticated FastAPI service with clean layers and database migrations.
- I can explain cache invalidation, secrets management, blob storage, and background ingestion.
- I can use evals, golden sets, and CI thresholds to prevent regressions.
- I can explain advanced RAG techniques such as HyDE, multi-query, parent-doc retrieval, reranking, and GraphRAG.
- I can defend multi-tenant SaaS architecture with tenant isolation, RLS, scoped tools, traces, and redacted logs.
- I can explain state correctness using transactions, idempotency, outbox, and avoiding dual writes.

Bottom line: the bootcamp was not only about calling an LLM. It taught the full AI engineering path: choose the right model, ground it with retrieval, let it act safely with tools, evaluate it continuously, package it as software, and protect production data with strong system boundaries.